Computer Vision Homework 7

Name: NICKHOLAS

Student ID: M11515801

Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement the thinning requirement.

Using OpenCV (cv2) only to read and write the image, and all of the processing is done on NumPy arrays with hand-written functions. Thinning is performed on a binarized lena.bmp that is downsampled from 512x512 to 64x64, using 4-connectivity. The thinning operator is built from three operators: the Yokoi connectivity number, the pair relationship operator, and the connected shrink operator.

Below are the results and explanations:



Explanation:

The preprocessing processes, binarization at the threshold 128 and downsampling image from 512x512 to 64x64 were conducted. A pixel is set to 255 if its gray value is at least 128, otherwise 0. The binarized image was divided into 8x8 blocks, and the topmost-left pixel of each block was taken as the downsampled data.

The image is then processed by the three-step operation iteratively. In each pass, the **Yokoi connectivity number** of the current image is computed first, the **pair relationship operator** is then applied to the Yokoi result to mark each pixel as p or q, and finally the **connected shrink operator** deletes pixels from the image. The passes are repeated until the image no longer changes.

Step 1: Yokoi connectivity number (4-connected)

Before the connectivity number function, below are the fragment of the codes for h function and f function. For each foreground pixel x0, the four corner neighborhoods (a1 to a4) are evaluated with h(b, c, d, e), where b is the center pixel, c is the 4-connected neighbor, and d and e are the remaining two pixels of that corner. The function returns q if b = c and (d != b or e != b), r if b = c and d = e = b, and s if b != c. Then f returns 5 if all four results are r (an interior pixel), otherwise the number of q among a1 to a4.

```
def h_func(b, c, d, e):
    if b == c:
        if d != b or e != b:
            return 'q'
        else:
            return 'r'
    else:
        return 's'
```

```
def f_func(a1, a2, a3, a4):
    if a1 == 'r' and a2 == 'r' and a3 == 'r' and a4 == 'r':
        return 5
    else:
        return [a1, a2, a3, a4].count('q')
```

```
def quadrant_labels(img, i, j):
    x0 = img[i][j]
    x1 = get_pixel(img, i, j + 1)      # right
    x2 = get_pixel(img, i - 1, j)      # top
    x3 = get_pixel(img, i, j - 1)      # left
    x4 = get_pixel(img, i + 1, j)      # bottom
    x5 = get_pixel(img, i + 1, j + 1)  # bottom-right
    x6 = get_pixel(img, i - 1, j + 1)  # top-right
    x7 = get_pixel(img, i - 1, j - 1)  # top-left
    x8 = get_pixel(img, i + 1, j - 1)  # bottom-left

    a1 = h_func(x0, x1, x6, x2)
    a2 = h_func(x0, x2, x7, x3)
    a3 = h_func(x0, x3, x8, x4)
    a4 = h_func(x0, x4, x5, x1)
    return a1, a2, a3, a4
```

```python
def yokoi_connectivity(img):
    h, w = img.shape
    result = np.full((h, w), -1, dtype=int)
    for i in range(h):
        for j in range(w):
            if img[i][j] == 0:
                continue
            a1, a2, a3, a4 = quadrant_labels(img, i, j)
            result[i][j] = f_func(a1, a2, a3, a4)
    return result
```

The pixels outside the image were treated as background (value 0), and the background pixels are not computed (they are marked as -1 in the result). The implementation details follow the course's lecture slides.

Step 2: Pair relationship operator

The pair relationship operator takes the Yokoi result as input. A foreground pixel is marked as p if its Yokoi connectivity number is 1 and at least one of its four 4-connected neighbors also has Yokoi connectivity number 1 (the threshold is 1); otherwise, it is marked as q. A pixel marked as p is only a candidate for deletion, and the pixels marked as q are always kept in this pass.

```python
def pair_relationship(yokoi):
    h, w = yokoi.shape
    marked = np.full((h, w), BACKGROUND, dtype=np.uint8)
    neighbors = [(0, 1), (-1, 0), (0, -1), (1, 0)]  # 4-neighbors
    for i in range(h):
        for j in range(w):
            if yokoi[i][j] == -1:
                continue
            count = 0
            for di, dj in neighbors:
                ni, nj = i + di, j + dj
                if 0 <= ni < h and 0 <= nj < w and yokoi[ni][nj] == 1:
                    count += 1
            if yokoi[i][j] == 1 and count >= 1:
                marked[i][j] = P
            else:
                marked[i][j] = Q
    return marked
```

Step 3: Connected shrink operator

The connected shrink operator is a recursive operator, so the image is scanned in raster-scan order and updated in place, and the decision of each pixel is made on the current (already updated) image. A pixel is deleted only if it satisfies both conditions: it is marked as p by the pair relationship operator, and it is removable by the connected shrink operator, i.e. exactly one of its four corner

results is 1, which is equal to a Yokoi connectivity number of 1. Deleting such a pixel does not disconnect the object. The marked image is computed once at the beginning of each pass from the image of that pass.

```
def can_shrink(img, i, j):
    a1, a2, a3, a4 = quadrant_labels(img, i, j)
    return f_func(a1, a2, a3, a4) == 1
```

```
def thinning_pass(img):
    h, w = img.shape
    yokoi = yokoi_connectivity(img)
    marked = pair_relationship(yokoi)

    changed = False
    for i in range(h):
        for j in range(w):
            if marked[i][j] != P:
                continue
            # evaluated on the current (already updated) image
            if img[i][j] != 0 and can_shrink(img, i, j):
                img[i][j] = 0
                changed = True
    return changed
```

```
def thinning(img):
    thin = img.copy()
    iterations = 0
    while thinning_pass(thin):
        iterations += 1
    return thin, iterations
```

Iterations

The three steps are repeated, and the output of each pass is used as the input of the next pass. In each pass, only the removable boundary pixels are peeled off, so the thick regions of the binarized image become thinner pass by pass, while the connectivity of every region is kept because a pixel is deleted only when its Yokoi connectivity number is 1. The iteration stops when a whole pass deletes no pixel, since the image would not change in any further pass.

The final result is the thinned 64x64 image shown at the beginning, in which the regions of the binarized image are reduced into thin lines of one pixel width while remaining connected.